Guidebook for the Lecture NLP with Deep Learning

TrustHLT

Dr. Prof. Ivan Habernal, Yassine Thlija

Winter Semester 2026/27

Guidebook to Exercise Sheet 1

This guide summarizes the concepts needed for Exercise Sheet 1. It is intended as a compact reference for classification evaluation, reference-based text generation metrics, and evaluation using large language models as judges.

1. True and Predicted Labels

For binary classification, predictions can be described using four fundamental quantities: true positives, true negatives, false positives, and false negatives.

In multiclass classification, these quantities can be defined separately for each class by temporarily treating one class as the positive class and all remaining classes as negative.

True Positive (TP)

Definition: The number of instances belonging to a class that are correctly predicted as that class.
Example: When considering `VB` as the positive class, the model predicts "verb" and the true label is also "verb".
Interpretation: A correct prediction for the class of interest.

True Negative (TN)

Definition: The number of instances that do not belong to the class of interest and are also not predicted as that class.
Example: When considering `VB`, a true noun is predicted as a noun.
Interpretation: The model correctly rejects the positive class.

False Positive (FP)

Definition: The number of instances incorrectly predicted as the class of interest.
Example: The model predicts `VB`, but the true label is `NN`.
Interpretation: The model predicts the positive class when it should not.

False Negative (FN)

Definition: The number of instances belonging to the class of interest that the model incorrectly predicts as another class.
Example: The true label is `VB`, but the model predicts `NN`.
Interpretation: The model misses a positive instance.

Term	Prediction	Actual	Interpretation
TP	Positive	Positive	Correct positive prediction
TN	Negative	Negative	Correct rejection
FP	Positive	Negative	Incorrect positive prediction
FN	Negative	Positive	Missed positive instance

2. Confusion Matrix

A **confusion matrix** summarizes classification performance by comparing true labels with predicted labels.

In this guide, and in the exercise sheet:

- rows represent the **true classes**,
- columns represent the **predicted classes**.

For example:

		Predicted class		
		`NN`	`VB`	`ADJ`
True class	`NN`	25	5	1
	`VB`	2	15	12
	`ADJ`	1	6	0

The diagonal contains correct predictions:

$$25 + 15 + 0 = 40.$$

The off-diagonal entries represent errors.

For a particular class i:

$$\begin{aligned} TP_i &= \text{diagonal entry for class } i, \\ FP_i &= \text{column sum for } i - TP_i, \\ FN_i &= \text{row sum for } i - TP_i. \end{aligned}$$

Example: class `VB`
The true positives are

$$TP_{\text{VB}} = 15.$$

The false positives are the non-verbs incorrectly predicted as verbs:

$$FP_{\text{VB}} = 5 + 6 = 11.$$

The false negatives are true verbs predicted as another class:

$$FN_{\text{VB}} = 2 + 12 = 14.$$

3. Classification Evaluation Metrics

Accuracy

Accuracy measures the fraction of all predictions that are correct:

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of predictions}}.$$

For the example confusion matrix:

$$\text{Accuracy} = \frac{25 + 15 + 0}{67} = \frac{40}{67} \approx 0.597.$$

Accuracy is intuitive, but can be misleading when the classes are strongly imbalanced.

Precision

Precision asks:

> Of all instances that the model **predicted as class** i, how many actually belong to class i?

$$P_i = \frac{TP_i}{TP_i + FP_i}.$$

High precision means that when the model predicts a class, it is often correct.

Recall

Recall asks:

> Of all instances that **actually belong to class** i, how many did the model correctly identify?

$$R_i = \frac{TP_i}{TP_i + FN_i}.$$

High recall means that the model misses relatively few examples belonging to the class.

F1 Score

F1 combines precision and recall using their harmonic mean:

$$F1_i = \frac{2P_iR_i}{P_i + R_i}.$$

F1 is high only when both precision and recall are high.

If both precision and recall are zero, the mathematical expression becomes

$$\frac{0}{0},$$

which is undefined. Evaluation libraries commonly define the resulting F1 score as zero according to their zero-division convention.

4. Micro, Macro, and Weighted Averaging

In multiclass classification, precision, recall, and F1 can be aggregated across classes in different ways.

Macro Averaging

Macro averaging treats every class equally.

For F1:

$$F1_{\text{macro}} = \frac{1}{K}\sum_{i=1}^{K} F1_i,$$

where K is the number of classes.

For three classes:

$$F1_{\text{macro}} = \frac{F1_{\text{NN}} + F1_{\text{VB}} + F1_{\text{ADJ}}}{3}.$$

A rare class therefore contributes just as much to macro-F1 as a very frequent class.

> **Important:**
> Standard macro-F1 is obtained by computing F1 for every class and then averaging those F1 scores.
> Computing macro precision and macro recall first and then taking their harmonic mean produces a different quantity. It should not be confused with standard macro-F1.

Micro Averaging

Micro averaging first aggregates TP, FP, and FN over all classes:

$$TP_{\text{micro}} = \sum_i TP_i,$$

and similarly for FP and FN.

Precision, recall, and F1 are then calculated from these global counts.

For a standard **single-label multiclass classification** problem,

$$P_{\text{micro}} = R_{\text{micro}} = F1_{\text{micro}} = \text{Accuracy}.$$

Weighted Averaging

Weighted averaging computes a metric for each class and weights each score according to the number of true examples of that class:

$$F1_{\text{weighted}} = \sum_{i=1}^{K} \frac{n_i}{N} F1_i,$$

where n_i is the number of true examples belonging to class i and N is the total number of examples.

Frequent classes therefore contribute more strongly than rare classes.

Average	Idea	Effect
Micro	Aggregate decisions before computing the metric.	Frequent classes dominate.
Macro	Compute per-class metrics and average equally.	Every class has equal importance.
Weighted	Compute per-class metrics and weight by class frequency.	Frequent classes contribute more.

5. Class Imbalance and the Accuracy Trap

Suppose a dataset contains

$$990$$

examples of class A and only

$$10$$

examples of class B.

A classifier that always predicts A achieves

$$\text{Accuracy} = \frac{990}{1000} = 99\%.$$

However, it detects none of the examples from class B.

This illustrates why a high accuracy does not necessarily imply that a classifier performs well for all classes.

For imbalanced datasets, it is often useful to inspect several quantities together:

- overall accuracy,
- macro-F1,
- per-class precision,
- per-class recall,
- per-class F1,
- and the confusion matrix itself.

A single number can hide important patterns of failure.

6. Implementing a Confusion Matrix

A confusion matrix can be implemented with a two-dimensional array.

Given

```
y_true = [...]
y_pred = [...]
labels = [...]
```

first map each label to an integer index.

For example:

```
label_to_idx = {
    "NN": 0,
    "VB": 1,
    "ADJ": 2
}
```

Then create an empty matrix:

```
matrix = np.zeros(
    (len(labels), len(labels)),
    dtype=int
)
```

For every pair of true and predicted labels, increment the corresponding entry:

```
for true_label, pred_label in zip(y_true, y_pred):
    i = label_to_idx[true_label]
    j = label_to_idx[pred_label]

    matrix[i, j] += 1
```

After constructing the matrix, the values needed for the metrics can be obtained using row sums, column sums, and diagonal entries.

For class i:

$$TP_i = C_{ii},$$

$$FP_i = \sum_j C_{ji} - C_{ii},$$

and

$$FN_i = \sum_j C_{ij} - C_{ii}.$$

Handling Zero Denominators

Evaluation code should explicitly handle cases such as

$$TP_i + FP_i = 0.$$

This means the model never predicted class i.

Likewise,

$$TP_i + FN_i = 0$$

means that class i does not occur among the true labels.

Do not allow these cases to silently produce unexpected values. Decide on a convention, such as assigning zero, and document that convention.

7. Part-of-Speech Tagging

Part-of-Speech (POS) tagging assigns a grammatical category to each word or token.

For example:

- `NN`: noun, such as *dog* or *computer*,
- `VB`: verb, such as *run* or *jump*,
- `ADJ`: adjective, such as *blue* or *heavy*.

For the purposes of this exercise, POS tagging is treated as a single-label multiclass classification problem: every evaluated token receives exactly one class.

8. Evaluating Text Generation

Classification usually has a finite set of possible labels.

Text generation is different.

For a question such as

> What is the capital of Germany?

several answers may express the same correct information:

- *Berlin.*
- *The capital of Germany is Berlin.*
- *Germany's capital city is Berlin.*

There may therefore be many valid outputs even when only one reference answer is available.

This makes generation evaluation fundamentally different from ordinary classification evaluation.

9. BLEU

BLEU (Bilingual Evaluation Understudy) was introduced primarily for evaluating machine translation.

It compares generated text with one or more reference texts using n-gram precision.

A simplified representation is

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right),$$

where:

- p_n is the modified precision for n-grams of size n,
- w_n is the weight assigned to n-grams of size n,
- BP is the brevity penalty.

Why a Brevity Penalty?

A system could artificially increase precision by generating extremely short outputs.

BLEU therefore includes a brevity penalty that penalizes outputs that are too short relative to the reference.

Advantages

- automatic and inexpensive,
- standardized,
- efficient for large datasets,
- historically useful for corpus-level machine translation evaluation.

Limitations

- depends strongly on lexical overlap,
- can penalize valid paraphrases,
- does not directly measure factual correctness,
- may behave poorly on individual short texts.

10. ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) is a family of reference-based evaluation measures historically associated with summarization.

Common variants include:

- **ROUGE-1**: unigram overlap,
- **ROUGE-2**: bigram overlap,
- **ROUGE-L**: overlap based on the longest common subsequence.

ROUGE measures how much information from a reference is recovered by the generated output.

Advantages

- inexpensive and automatic,
- intuitive,
- useful for measuring reference-content coverage,
- widely used for summarization.

Limitations

- relies on surface overlap,
- may penalize valid paraphrases,
- does not directly measure factual consistency,
- does not guarantee fluency or coherence.

11. The Limits of Lexical Overlap

Consider the reference:

> *Berlin is the capital of Germany.*

Now consider:

> **A:** *Berlin is the capital of Germany.*
>
> **B:** *Germany's capital is Berlin.*
>
> **C:** *Berlin is not the capital of Germany.*

Output B is correct despite changing the surface form.

Output C is incorrect despite sharing almost all words with the reference.

This exposes a central limitation of lexical-overlap metrics:

> **Lexical similarity is not the same thing as semantic correctness.**
> Two texts can have different wording but the same meaning. Conversely, a small change in wording can completely reverse the meaning of a sentence.

12. LLMs as Judges

One approach to evaluating open-ended generation is to use another large language model as an evaluator.

This approach is commonly called

LLM-as-a-Judge.

Instead of asking how many words overlap with a reference, we can ask a judge model to assess properties such as:

- factual correctness,
- relevance,
- completeness,
- coherence,
- conciseness,
- instruction following.

For example:

> **Question**
> Why do leaves appear green?
>
> **Candidate answer**
> Leaves appear green because chlorophyll absorbs much of the red and blue light while reflecting more green light.
>
> **Judge instruction**
> Evaluate the factual correctness of the answer on a scale from 1 to 5.

13. Pointwise Evaluation

In **pointwise evaluation**, the judge evaluates one candidate answer at a time.

For example:

> Evaluate the following answer for factual correctness.
> Assign a score from 1 to 5, where:
>
> - 1 = completely incorrect,
> - 2 = mostly incorrect,
> - 3 = partially correct,
> - 4 = mostly correct,
> - 5 = completely correct.

This produces an absolute score for each answer.

Pointwise evaluation is convenient when many individual outputs need to be scored independently.

However, numerical scores can be difficult to calibrate consistently. For example, the difference between a score of 3 and 4 may not be interpreted identically across questions or judges.

14. Pairwise Evaluation

In **pairwise evaluation**, two candidate answers are shown to the judge simultaneously.

For example:

> Given the question and the two candidate answers below, determine which answer is more factually correct.
> Return exactly one of:
>
> `A`, `B`, `Tie`.

Instead of assigning absolute scores, the judge answers:

$$A > B, \qquad B > A, \qquad \text{or} \qquad A = B.$$

Pairwise evaluation can make the task easier because the judge only needs to determine which response better satisfies the evaluation criteria.

However, comparing many systems may require many comparisons.

	Pointwise	**Pairwise**
Input	One response	Two responses
Output	Usually a score or label	A, B, or Tie
Main advantage	Simple individual scoring	Direct comparison
Potential issue	Score calibration	Ordering and position effects

15. Designing a Judge Rubric

A vague instruction such as

> Is this answer good?

leaves the meaning of "good" unspecified.

A stronger evaluation prompt explicitly defines the criteria.

For example:

> Evaluate the answer according to:
>
> 1. **Factual correctness:** Are the factual claims correct?
> 2. **Relevance:** Does the response address the question?
> 3. **Completeness:** Does the response contain the important information needed to answer the question?
>
> Score each criterion from 1 to 5.

Clear criteria reduce ambiguity and make the evaluation protocol easier to reproduce.

16. Biases of LLM Judges

An LLM judge is still a model. Its judgments should therefore not automatically be treated as ground truth.

Position Bias

In pairwise evaluation, the judge may systematically prefer the answer shown first or second.

Suppose the judge receives:

$$(A = X,\ B = Y)$$

and chooses A.

After swapping the order:

$$(A = Y,\ B = X),$$

it again chooses A.

This suggests that the position of the response, rather than only its content, influences the judgment.

A simple diagnostic is therefore to evaluate both orders.

Verbosity Bias

A judge may prefer longer and more detailed answers even when the additional content does not improve correctness.

For example, a long answer and a concise answer may contain exactly the same relevant information, yet the longer answer may receive a higher score.

Style Bias

A fluent, confident, or professionally written answer may appear better even if it contains a factual error.

The judge may therefore accidentally reward presentation quality when it is supposed to evaluate factual correctness.

Self-Preference

A model may systematically prefer outputs generated by itself or by models with similar characteristics.

This can create an unfair advantage when one model family is used both as a generator and as the evaluator.

Prompt Sensitivity

Judgments may change when the evaluation prompt changes.

For example:

> Rate the factual correctness of this answer.

and

> Act as a strict evaluator. Carefully identify every possible factual error before assigning a score.

may produce different judgments for exactly the same candidate answer.

17. Reducing Judge Bias

Several techniques can make LLM-based evaluation more robust.

Swap Candidate Order

For pairwise evaluation, evaluate

$$(A, B)$$

and

$$(B, A).$$

A decision is more convincing if the preferred response remains the same after correcting for the swapped labels.

Use Explicit Rubrics

Define exactly what the judge should evaluate.

Avoid relying on vague concepts such as “quality” unless they are carefully defined.

Use Multiple Judges

Evaluating the same outputs with multiple models can reveal whether a result depends strongly on one particular judge.

Judges from different model families may be especially informative.

Control Irrelevant Factors

When possible, test whether factors unrelated to the target criterion influence judgments.

Examples include:

- answer length,
- writing style,
- answer order,
- model identity,
- formatting.

18. Validating an LLM Judge with Humans

LLM judges should ideally be validated against human judgments.

Suppose an LLM evaluates 500 answers.

Human annotators independently evaluate a subset of 100 answers, and the LLM agrees with the human majority on 78.

Then:

$$\text{Agreement} = \frac{78}{100} = 78\%.$$

This provides useful evidence, but it is not sufficient by itself to establish that the judge is reliable.

Human–Human Agreement

Human annotators may disagree with one another.

Therefore, it is also important to measure agreement between humans.

If humans themselves frequently disagree, the evaluation task may be ambiguous or subjective.

Human–human agreement provides useful context for interpreting human–LLM agreement.

Inspect the Disagreements

Aggregate agreement can hide systematic failures.

For example, an LLM judge might perform well overall but consistently misjudge:

- very long answers,
- answers involving numerical reasoning,
- particular domains,
- outputs from a particular model,
- answers containing subtle factual errors.

Therefore, disagreements should be inspected rather than summarized only by one percentage.

19. A Practical LLM-as-a-Judge Pipeline

A reasonable evaluation pipeline might look as follows:

1. **Define the evaluation criteria.**
   Decide what "better" means before evaluating the models.
2. **Choose pointwise or pairwise evaluation.**
   Pointwise evaluation produces individual scores, while pairwise evaluation directly compares candidate responses.
3. **Construct a clear judge prompt.**
   Specify the task, criteria, output format, and scoring rubric.
4. **Control ordering effects.**
   For pairwise evaluation, consider evaluating candidates in both orders.
5. **Evaluate with one or more judge models.**
   Multiple judges can help identify judge-specific effects.
6. **Validate against humans.**
   Human-evaluate a representative subset of the data.
7. **Measure agreement.**
   Examine both human–human and human–judge agreement.
8. **Inspect systematic disagreements.**
   Determine whether particular response types are consistently favored or penalized.
9. **Aggregate results.**
   Depending on the protocol, report average scores, win rates, ties, agreement statistics, or rankings.
10. **Report the evaluation protocol.**
    Document the judge model, prompt, rubric, ordering procedure, and aggregation method.

Key principle
An LLM judge does not produce an objective measurement simply because it is a powerful model.
The final result depends on the judge model, prompt, rubric, evaluation format, aggregation procedure, and validation protocol.
Therefore, LLM-as-a-Judge should be treated as an **evaluation methodology** whose assumptions and limitations must be made explicit.